

Exercícios — Aula 06

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

⚠ **Estes exercícios são pré-requisito para a aula 07.** Chegue com a refatoração pronta, porque na próxima aula vamos trocar o repository por um banco de dados — e isso só é simples se as camadas existirem.

O objetivo não é escrever código novo: é **reorganizar** o que você já fez. Ao final, a API de filmes deve se comportar **exatamente igual** — mesmos endpoints, mesmos status, mesmas respostas.

Nível 1 — obrigatório

Exercício 1 — Criar a estrutura

Reorganize o projeto `api-filmes` nesta estrutura:

```
api-filmes/
├── .env
├── .env.example
├── .gitignore          (com node_modules/ e .env)
├── package.json
├── api.http
└── src/
    ├── server.js
    ├── app.js
    ├── routes/filme.routes.js
    ├── controllers/filme.controller.js
    ├── services/filme.service.js
    ├── repositories/filme.repository.js
    ├── validacoes/filme.validacao.js
    ├── middlewares/
    │   ├── tratadorDeErros.js
    │   └── rotaNaoEncontrada.js
    └── erros/index.js
```

Faça o servidor subir lendo a porta do `.env`:

```
"scripts": { "dev": "node --watch --env-file=.env src/server.js" }
```

Dica de método: mova uma camada por vez, de baixo para cima (repository → service → controller → routes), rodando a API depois de cada passo. Não tente mover tudo de uma vez.

Exercício 2 — Repository e Service

- `filme.repository.js` — só acesso a dados: `listarTodos`, `buscarPorId`, `buscarPorTituloEAno`, `criar`, `atualizar`, `remover`. Nenhuma regra, nenhum `throw`.
- `filme.service.js` — todas as regras: validação, duplicidade (409), filtros de listagem, montagem do objeto novo.

Teste seu entendimento: procure por `req`, `res` ou `status` dentro do `service.js`. Se encontrar algum, a separação ainda não está certa.

Exercício 3 — Controller, Routes e erros

- `erros/index.js` — `ErroDaAplicacao`, `ErroNaoEncontrado`, `ErroDeValidacao`, `ErroDeConflito`.
- `filme.controller.js` — cada método com `try/catch` e `next(erro)`.
- `filme.routes.js` — só o mapa das rotas, com `Router()`.
- `tratadorDeErros.js` — o único lugar que transforma erro em status HTTP.

No final, rode **todo** o seu `api.http` da aula 05 e confirme: mesmos status, mesmas respostas. Se alguma coisa mudou, a refatoração introduziu um bug.

Nível 2 — esperado

Exercício 4 — Middleware de validação de id

Hoje cada rota com `:id` repete a conversão e a checagem. Extraia isso para um middleware:

```
// middlewares/validarId.js
export function validarId(req, res, next) {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id < 1) {
    return next(new ErroDeValidacao(["o id precisa ser um número inteiro positivo"]));
  }
  req.idNumerico = id; // deixa pronto para o controller
  next();
}
```

Use assim:

```
router.get("/:id", validarId, filmeController.buscarPorId);
```

Repare que uma rota pode ter **vários** middlewares em fila — eles rodam na ordem em que aparecem.

Exercício 5 — Segundo recurso

Adicione o recurso **avaliações** seguindo exatamente a mesma estrutura de camadas:

```
avaliacao.routes.js, avaliacao.controller.js, avaliacao.service.js,  
avaliacao.repository.js.
```

No `app.js`:

```
app.use("/filmes", filmeRoutes);  
app.use("/avaliacoes", avaliacaoRoutes);
```

Regra de negócio (vive no **service** de avaliação): não é possível avaliar um filme que não existe. O service de avaliação pode chamar o service de filme para verificar.

Este exercício é a prova real da arquitetura: se adicionar um recurso novo foi rápido e mecânico, a estrutura está certa.

Desafio — opcional

Exercício 6 — Validação com Zod

Substitua a validação manual pela biblioteca [Zod](https://zod.dev) (<https://zod.dev>):

```
npm install zod
```

```
import { z } from "zod";

export const esquemaNovoFilme = z.object({
  titulo: z.string().min(1, "titulo é obrigatório").max(200),
  ano: z.number().int().min(1888).max(new Date().getFullYear() + 5),
  genero: z.enum(["drama", "acao", "comedia", "suspense", "terror", "documentario"]),
  nota: z.number().min(0).max(10).optional(),
  assistido: z.boolean().optional(),
});

// no service:
const resultado = esquemaNovoFilme.safeParse(dados);
if (!resultado.success) {
  throw new ErroDeValidacao(resultado.error.issues.map((problema) => problema.message));
}
const dadosLimpos = resultado.data;
```

Compare o seu `validacao.js` antes e depois. Escreva num comentário: o que melhorou e o que você perdeu ao trocar código próprio por biblioteca?

Exercício 7 — Configuração centralizada

Crie `src/config/index.js` que lê todas as variáveis de ambiente, valida que as obrigatórias existem e exporta um objeto pronto:

```
export const config = {
  porta: Number(process.env.PORTA ?? 3000),
  ambiente: process.env.NODE_ENV ?? "desenvolvimento",
  ehProducao: process.env.NODE_ENV === "producao",
};
```

Se uma variável obrigatória faltar, o programa deve **falhar ao iniciar** com uma mensagem clara — e não descobrir o problema só quando a rota for chamada. Isso se chama *fail fast*.

Checklist de entrega

- ☐ Nenhum `req`, `res` ou status code aparece dentro de `services/`
- ☐ Nenhuma regra de negócio aparece dentro de `repositories/`
- ☐ Todo controller tem `try/catch` com `next(erro)`
- ☐ O `tratadorDeErros` é o **último** `app.use` do `app.js`
- ☐ `.env` está no `.gitignore` e existe um `.env.example` versionado

- ☐ Rodei o `api.http` inteiro e o comportamento é idêntico ao da aula 05
- ☐ Consigo responder, sem pensar muito: "onde mexo para mudar uma regra de negócio?"